
title: "Plumb — Visual Flowcharts"

subtitle: "AWS Architecture with Real Diagrams"

author: "Engineering Team · Plumb"

date: "August 2026"

geometry: "margin=2cm,top=2.5cm"

fontsize: 11pt

mainfont: "Helvetica"

monofont: "Menlo"

\newpage

Plumb — Visual Flowcharts

This document complements ``PLUMB-SYSTEM-ARCHITECTURE.md`` with rendered

flowcharts. Each diagram is generated from a Mermaid source file in

``docs/flowcharts/`` and rendered to SVG via ``@mermaid-js/mermaid-cli``.

The diagrams use the **Plumb design system colors** per ``packages/tokens``:

- **Teal #00B894** — primary action, control plane, services
- **Amber #F5A524** — warnings, storage, GPU, pipelines
- **Light-teal #4FDDDB** — success states, stable deployments
- **Dark #0A0D13** — backgrounds, regions
- **Light #F2F4F7** — text on dark

\newpage

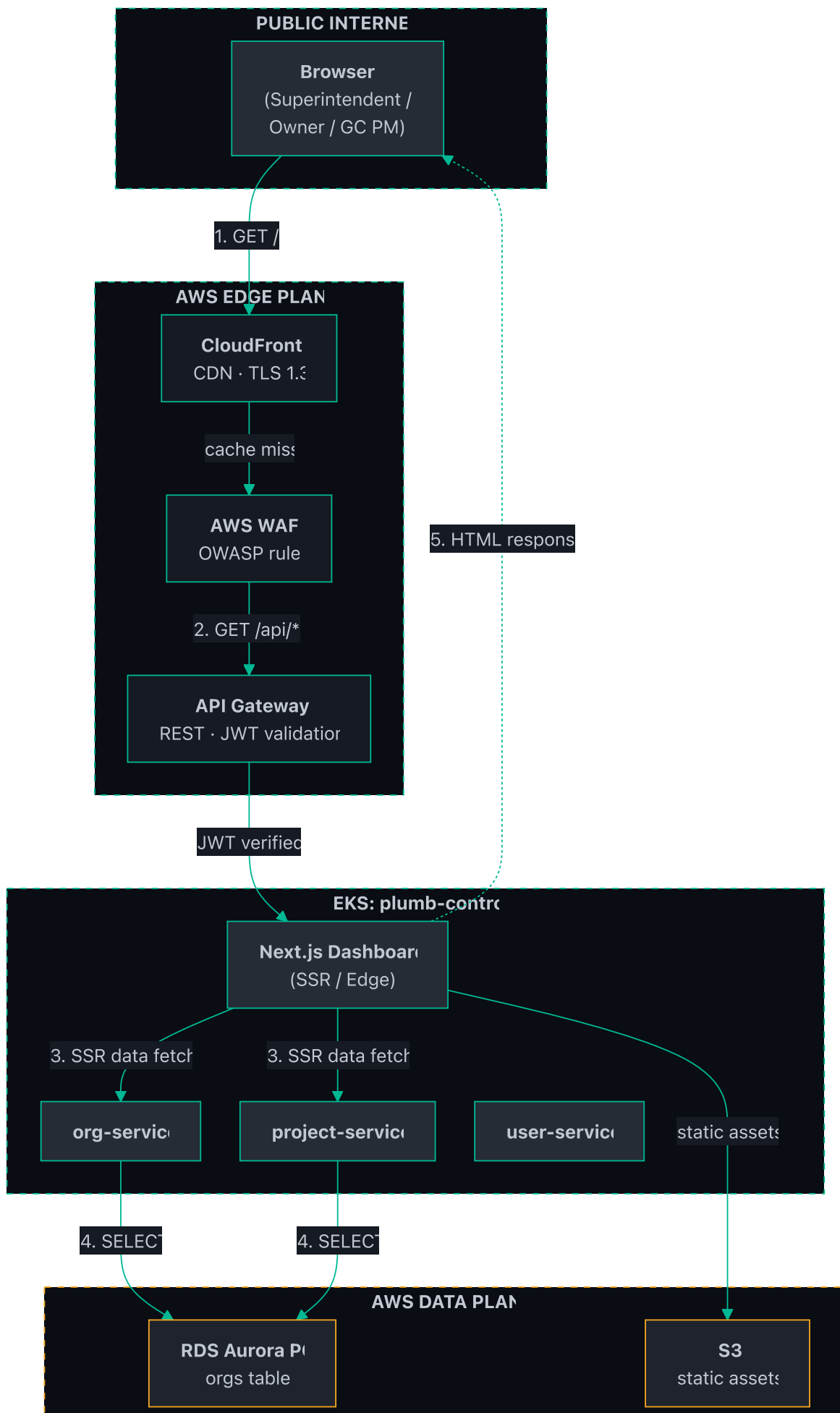
1. User Opens the Dashboard

A VDC PM lands on ``https://app.plumb.dev``. CloudFront serves cached static

assets from S3; uncached API requests fan through WAF → API Gateway →

Next.js SSR → the relevant backend services. The org-service and

project-service are called in parallel from the page component.



****Key properties:****

- All CloudFront → ALB traffic is HTTPS (TLS 1.3)
- API Gateway validates the Cognito JWT on every request
- `x-request-id` is propagated through every hop (per `@plumb/observability`)
- Next.js SSR fetch uses `force-dynamic` so the home page is always fresh
- HTML is CDN-cached for 60s; API responses are not cached

\newpage

2. Capture Initiation + Chunked Upload + Async Pipeline

A superintendent in the field starts a 360° walk. The mobile app initiates through the API, uploads chunks directly to S3, and an async Step Functions state machine runs the spatial AI pipeline on GPU nodes.

📱 iPhone + Insta360 X4
(AVCaptureSession record
8K equirectangular @ 30fp
CoreMotion fuses IMU + GP)

Phase 1 — Capture Initiation

1. POST
/v1/projects/{id}/captures
Bearer Cognito JWT
Idempotency-Key: uuid-per-session

2. API Gateway validates JWT
+ forwards to capture-service

3. INSERT captures row
(status='uploading')

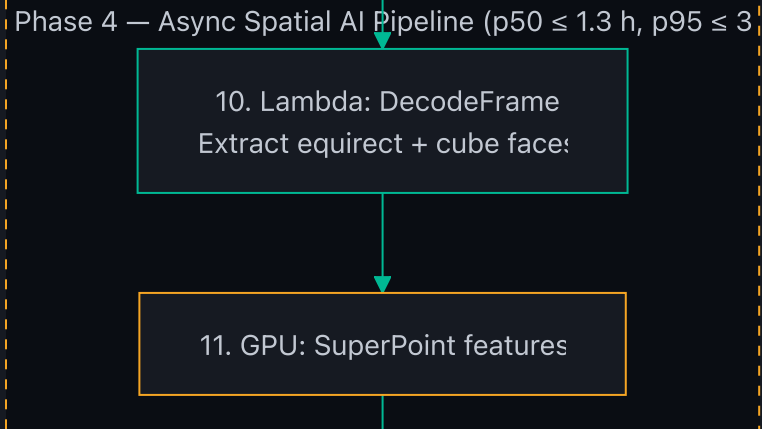
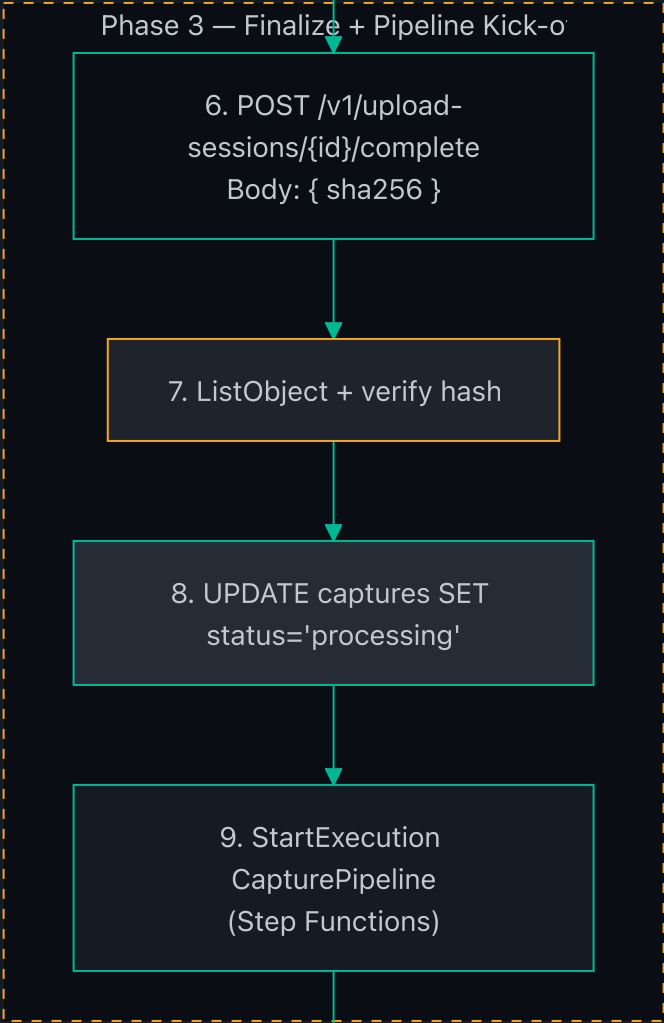
4. Return upload session
{id: 'upl_xxx', chunkUrls: [...]}

Phase 2 — Chunked Upload (direct to S3)

5a. PUT chunk 0 (8 MB,
signed URL)

5b. PUT chunk 1

5c. PUT chunk N
(retained on retry;
idempotent at chunk level)



12. GPU: SfM
(COLMAP/GLOMAP)

13. GPU: MVS (OpenMVS)

14. GPU: Poisson mesh

15. GPU: 3D Gaussian Splatting

16. GPU: SAM-2 segmentation

17. GPU: Align to BIM (DINOv2
+ ICP)

Phase 5 — Publish

18. StateMachineSucceeded
+ artifact URLs (S3, tile paths)

19. UPDATE captures SET
status='ready'

```
20. WebSocket push (via
    realtime-gateway)
    { type: 'capture.ready',
      artifacts: [...] }
```

Key properties:

- **Connection resilience:** mobile uses `NSURLSession.background`` (iOS) and `WorkManager`` (Android) so uploads survive airplane-mode drops
- **Direct-to-S3 chunking:** the mobile app gets pre-signed S3 URLs and uploads chunks directly, bypassing the API gateway
- **Idempotency:** `Idempotency-Key = uuid-per-session`` means the mobile can retry the entire initiation on flaky network without creating duplicate captures
- **Pipeline is async:** the mobile gets back immediately with an upload session; spatial AI runs in the background on GPU nodes

\newpage

3. Spatial AI Pipeline — Step-by-Step

The eight sequential stages of the GPU pipeline. Each stage is a

Step Functions state; failures route to a DLQ.



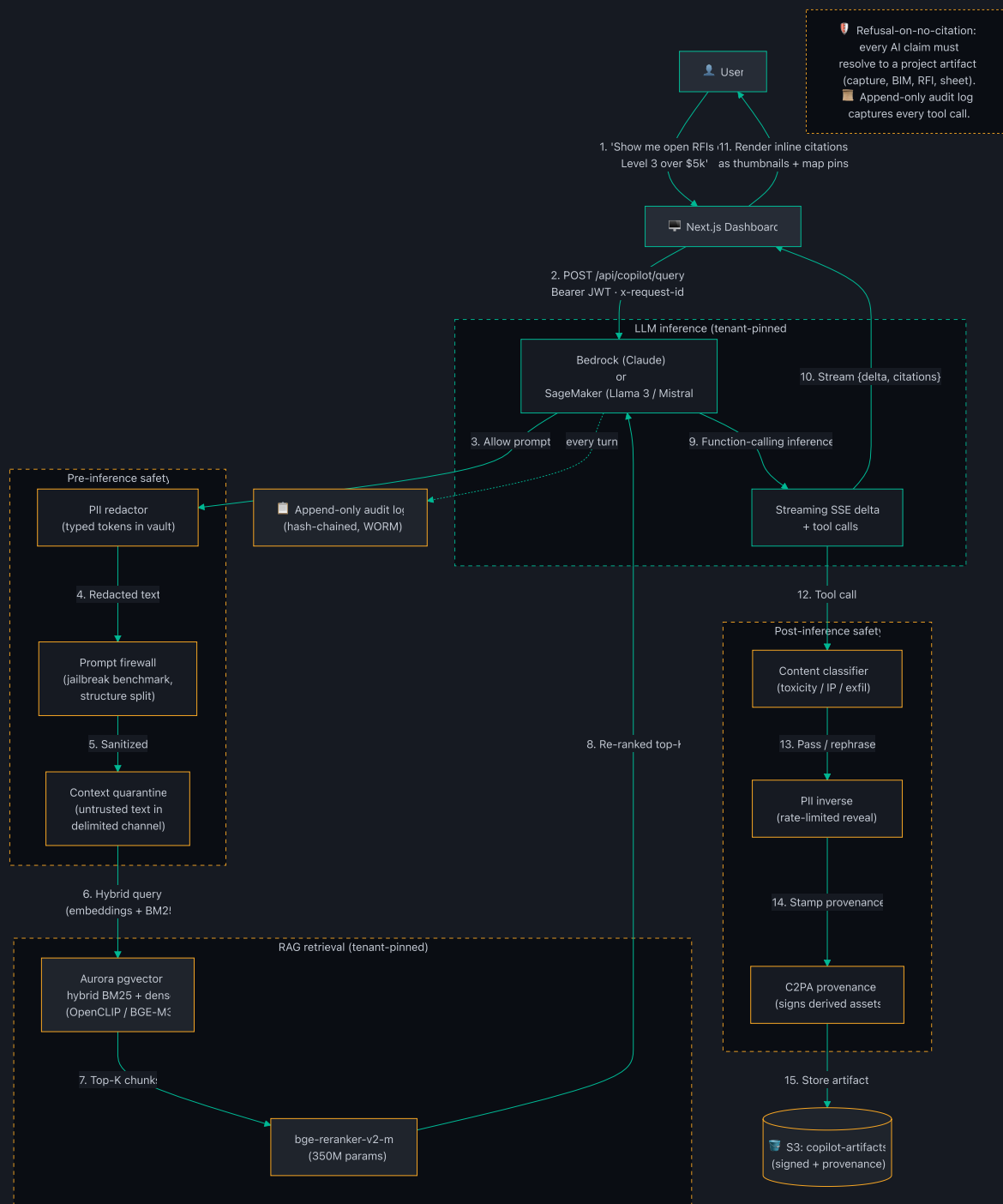
Stage details:

- **DecodeFrame** — Lambda function; produces equirectangular + cube faces
- **SuperPoint** — per-frame keypoint extraction (Gilles Simonin / MS)
- **SfM (COLMAP/GLOMAP)** — recovers camera poses and sparse point cloud
- **MVS (OpenMVS)** — dense depth maps via PatchMatch + multi-view
- **Poisson** — screened Poisson surface reconstruction
- **3D Gaussian Splatting** — photoreal novel-view (per-room scenes)
- **SAM-2** — semantic segmentation (per-trade masks)
- **Align to BIM** — DINOv2 + ICP for registry

\newpage

4. AI Copilot Query — Safety Pipeline

A user types "Show me open RFIs on Level 3 over \$5k". The request flows through a pre-inference safety pipeline, RAG retrieval, LLM inference, and a post-inference safety pipeline before returning citations to the user.



****Key properties:****

- ****Safety pipeline runs before AND after the model**** (PII redactor on the way in, content classifier on the way out)
- ****C2PA provenance**** stamped on every derived asset
- ****Citations are mandatory**** — refusal-on-no-citation policy
- ****Tenant-locked retrieval**** — per-tenant key prefixes enforced at the

vector store level

- **Streaming SSE** keeps the browser responsive even for multi-second retrieval-then-reason cycles

\newpage

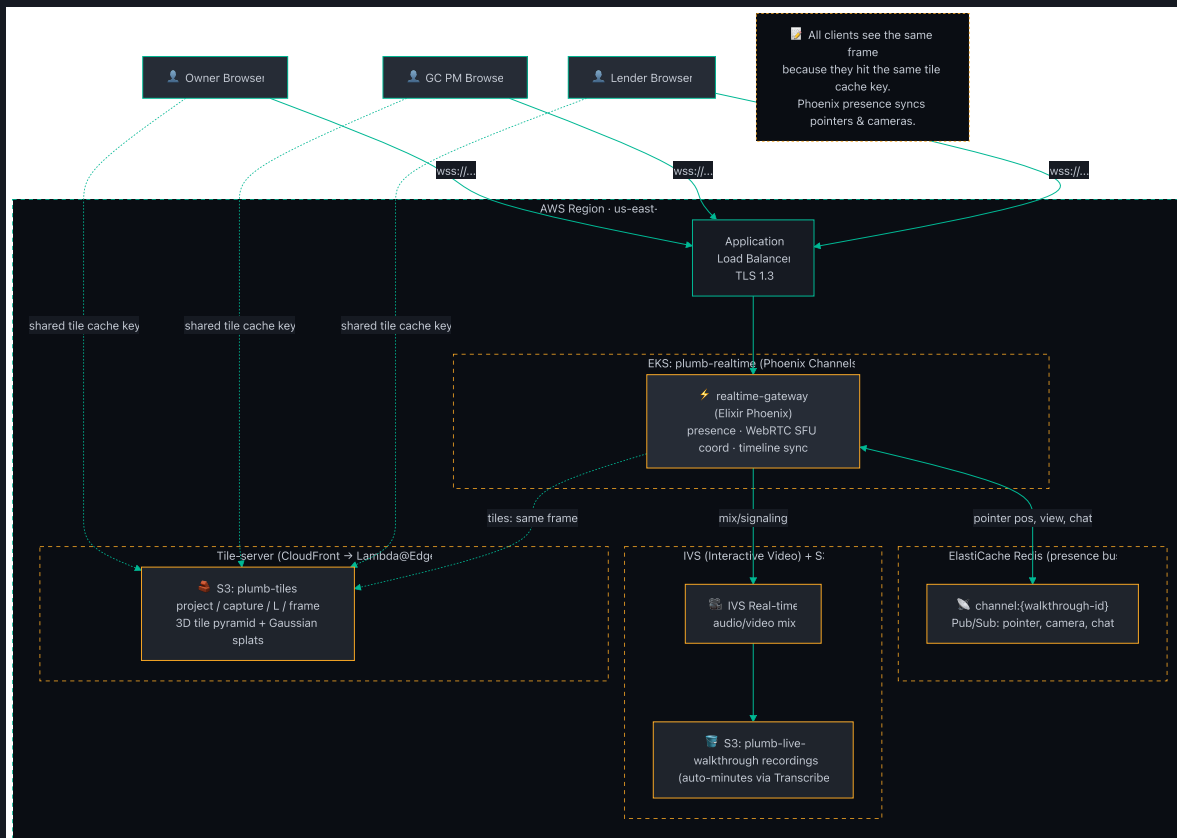
5. Multi-Stakeholder Live Walkthrough

The owner, the GC PM, and a lender are all in the same live 360°

walkthrough. They share a tile cache key so the same frame is rendered

on every screen, while presence + audio/video are routed through Phoenix

Channels + IVS.

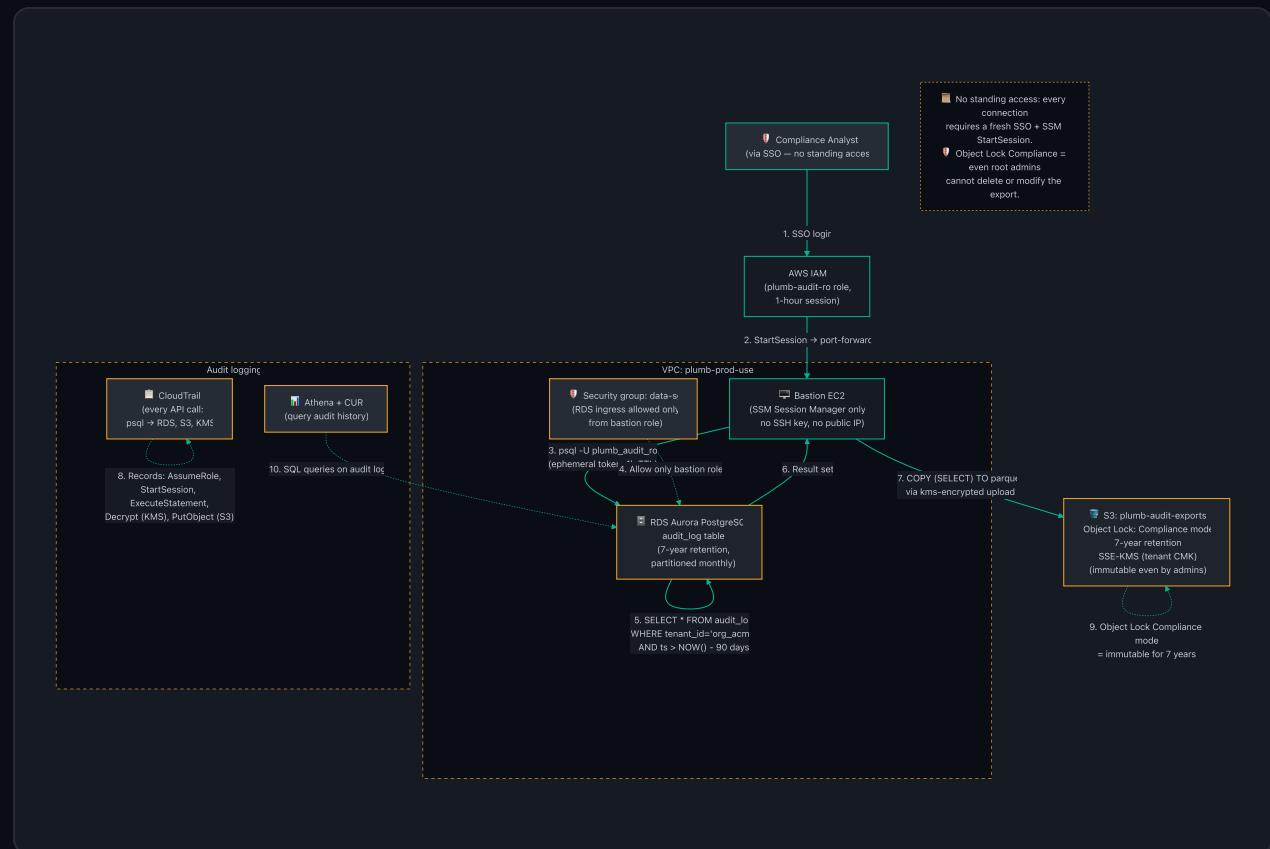


Key properties:

- Phoenix Channels** for soft-realtime presence (BEAM VM's fault tolerance and natural fan-out)
- IVS (Interactive Video Service)** for real-time audio/video
- Redis Pub/Sub** as the presence bus (channel-per-walkthrough)
- S3** for recordings; auto-minutes via Transcribe → Bedrock summary
- CloudFront + Lambda@Edge** for signed tile URLs per viewer

6. SOC 2 Audit Query

The compliance team needs a query to answer "show every access to a record owned by ACME Corp in the last 90 days". Admins never have standing access — every connection is via SSO + SSM Session Manager.



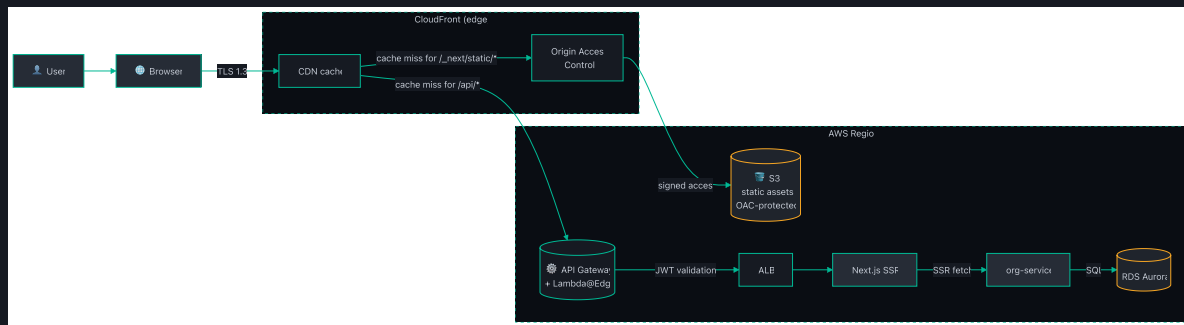
****Key properties:****

- **No standing access:**** Admins use SSM Session Manager, never long-lived credentials
- **CloudTrail**** records every API call (including psql queries, which generate RDS API calls)
- **Object Lock + Compliance mode**** prevents audit export tampering
- **DB-level role separation**** (`plumb\_audit\_ro`) prevents admins from masquerading as application users
- **Tamper-evident audit log**** — every entry has a hash chain anchored to S3

7. Edge CDN Architecture

The full edge path: CloudFront smart-routing, Origin Access Control to S3,

WAF, API Gateway, ALB, Next.js SSR, and the underlying services.



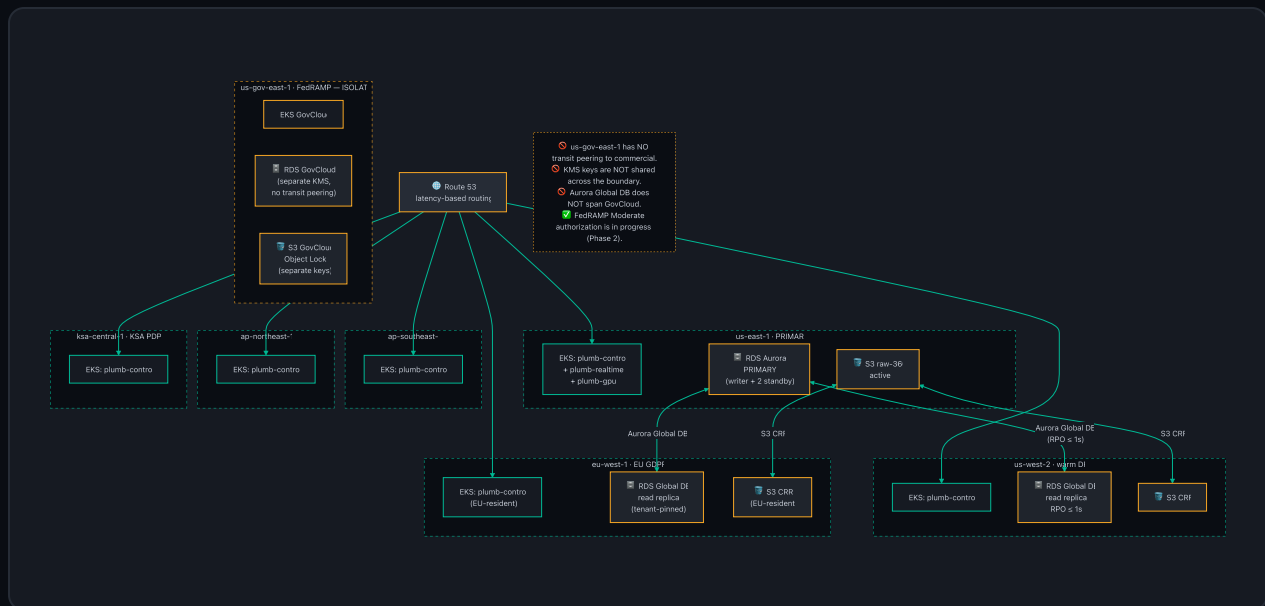
Key properties:

- **OAC** (Origin Access Control) protects S3 from public access
- **CloudFront** serves `/api/*` from ALB and `/_next/static/*` from S3
- **Lambda@Edge** for JWT validation at the edge (saves a round trip to the origin)
- **TLS 1.3** everywhere; ACM-managed certs auto-rotated

\newpage

8. Multi-Region Topology

Seven regions: six commercial (us-east-1 primary, us-west-2 warm DR, eu-west-1 EU-resident, ap-southeast-2, ap-northeast-1, ksa-central-1) and one isolated GovCloud (us-gov-east-1) for FedRAMP.



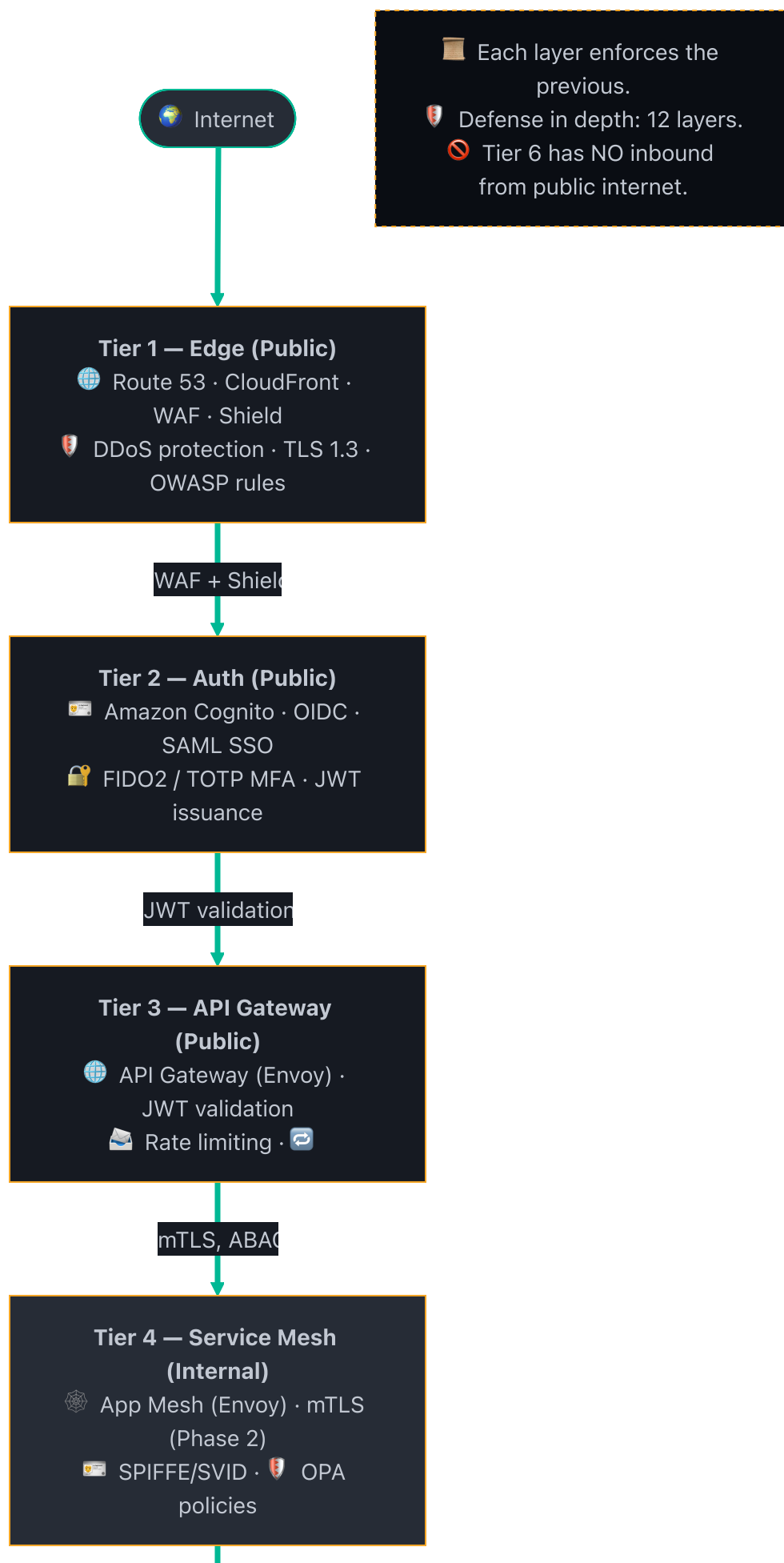
****Key properties:****

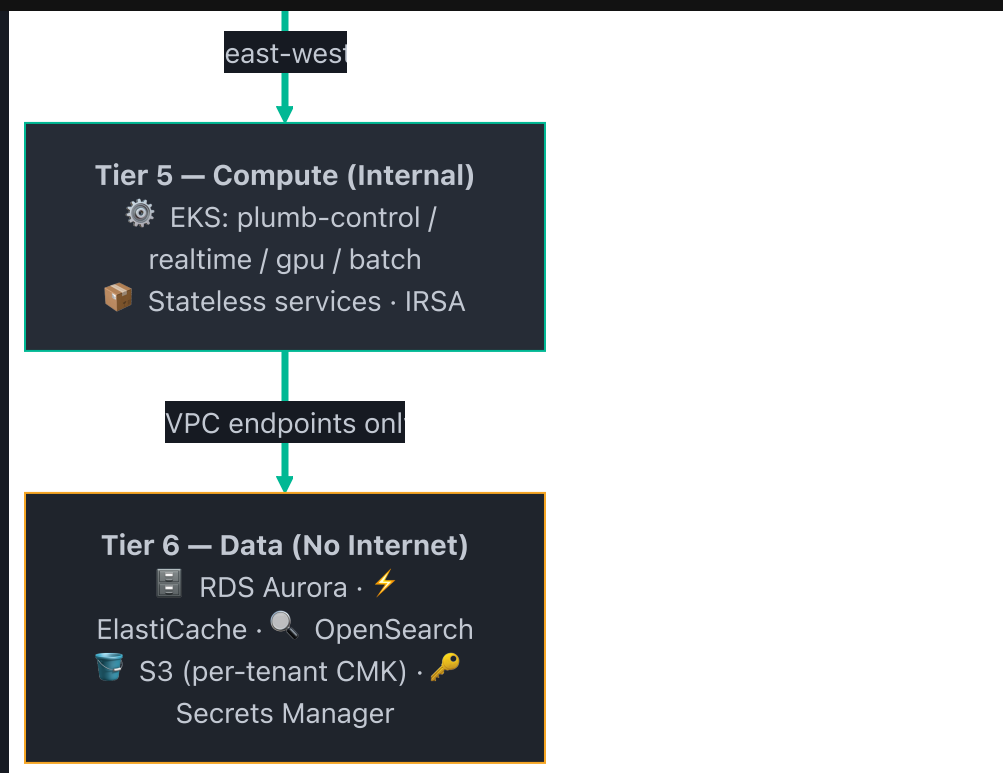
- ****Aurora Global DB**** for cross-region read replicas (RPO ≤ 1s)
- ****S3 Cross-Region Replication**** for raw-360 and audit-exports
- ****Route 53 latency-based routing**** across regions
- ****Transit Gateway peering**** between commercial regions
- ****GovCloud isolated**** — no transit peering to commercial, no shared KMS

\newpage

9. Defense in Depth — 6 Layers

Every request passes through six trust boundaries. Each layer enforces the previous and rejects what should not pass.





****Key properties:****

- ****12 layers total**** (the 6 above plus 6 application-layer defenses)
- ****Tier 6 has NO inbound from public internet**** — only VPC endpoints
- ****Per-layer SLO burn alerts**** so on-call knows exactly which layer is failing without guessing
- ****mTLS for east-west**** traffic (Phase 2 SPIFFE/SVID)

\newpage

10. Canary Deployment via Argo Rollouts

A new version of `org-service` rolls out via Argo Rollouts with SLO-gated progression. If SLO burn exceeds 2x at any step, auto-rollback to 100% stable.



****Step progression:**** 1% → 5% → 25% → 50% → 100%, with 5–20 minute pauses between each step. ****Hard guard:**** SLO burn > 2x triggers instant rollback to the last known-good revision (preserves last 3 revisions for forensic diff).

Appendix: Source Files

Every diagram in this document is generated from a source file in

`docs/flowcharts/`. To regenerate:

```
export PATH="/tmp/mermaid-render/node_modules/.bin:$PATH"
mmdc -i docs/flowcharts/01-dashboard.mmd -o docs/flowcharts/01-dashboard.svg
```

Or all at once:

```
for f in docs/flowcharts/*.mmd; do
  name=$(basename "$f" .mmd)
  mmdc -i "$f" -o "docs/flowcharts/${name}.svg" --backgroundColor transparent
done
```

Appendix: Color Reference

Token	Hex	Used for
-------	-----	----------

---	---	---
-----	-----	-----

`signal500`	#00B894	Primary action, control plane, services
-------------	---------	---

`signal300`	#4FDDB6	Success states, stable deploys
-------------	---------	--------------------------------

`amber500`	#F5A524	Warnings, storage, GPU, pipelines
------------	---------	-----------------------------------

`ink950`	#0A0D13	Background, regions
----------	---------	---------------------

`ink900`	#161A22	Surface 1 (raised)
----------	---------	--------------------

`ink800`	#262C36	Surface 2 (overlay)
----------	---------	---------------------

`ink100`	#F2F4F7	Text on dark
----------	---------	--------------

End of document.